

Project Rules and Standards

Coding, security, workflow, and business rules everyone follows

Project	Course Enrollment Platform (courses: Digital Marketing, SEO, Content Writing, Web Development)
Version	1.0 (Draft for developer review)
Audience	Development team, project owner, and QA

These rules keep the code clean, the site secure, and the business logic consistent. Where a rule cannot be followed, the developer must record the reason in the pull request.

1. Business Rules

ID	Rule
BR-01	Every course has at least one published batch before its Enroll button is shown; otherwise show "Notify me for next batch".
BR-02	A batch never exceeds its capacity. When seats_taken equals capacity, status becomes Full.
BR-03	A student can hold only one active enrollment per batch.
BR-04	New enrollments are Pending. Only an admin approval (manual) or a verified gateway webhook (online) can make them Confirmed.
BR-05	A pending enrollment holds a seat for 48 hours. After that the seat is released automatically and the student is emailed.
BR-06	Fees are stored in PKR as integers; discounts are applied before payment and recorded on the enrollment.
BR-07	Refunds follow the published Refund Policy and are recorded as a Payment with status Refunded, with the admin who processed it.
BR-08	Reviews are allowed only for Confirmed or Completed enrollments and appear after admin approval.
BR-09	Certificates are issued only for Completed enrollments and each has a unique verification code.
BR-10	Deleting a course with enrollments is not allowed; it can only be unpublished (archived).
BR-11	Student personal data is used only for admissions and communication and is never shared with third parties.

2. Coding Standards

- TypeScript with strict mode on, for both frontend and backend. No use of **any** without a comment explaining why.
- ESLint and Prettier are mandatory; code must pass lint with zero errors before a pull request.
- Naming: **camelCase** for variables and functions, **PascalCase** for components and classes, **kebab-case** for file names and URLs, **UPPER_SNAKE** for constants.
- One component or class per file; keep functions short and single-purpose.
- No hard-coded strings for fees, statuses, or URLs; use constants and environment variables.
- No dead code, commented-out blocks, or console.log left in merged code.

- Every function that talks to the database or an external service handles errors and returns a clear message.
- Reusable UI (buttons, cards, forms, modals) lives in the shared components folder and is not duplicated.

3. Git and Workflow Rules

Topic	Rule
Branches	main (production), develop (staging), and feature branches named feature/short-name, fix/short-name, or chore/short-name
Commits	Small, focused, present-tense messages, e.g. "Add batch capacity check". Format: type: message (feat, fix, docs, refactor, test, chore)
Pull requests	Every change goes through a pull request into develop; at least one review is required; the description states what changed and how it was tested
Merging	No direct pushes to main or develop; squash merge after CI passes
Releases	Tagged versions (v1.0.0, v1.0.1); a short changelog for every release
Secrets	Never commit .env files, keys, or passwords; use .env.example with placeholder values

4. Security Rules

- Passwords: minimum 8 characters, hashed with bcrypt; never stored or logged in plain text.
- Every protected route checks the user's role on the server; hiding a button in the UI is not security.
- All inputs are validated and sanitized on the server, even if the frontend already validated them.
- Use parameterized queries through the ORM; raw SQL requires review approval.
- Uploads: only JPG, PNG, and PDF, maximum 5 MB, random file names, stored outside the public web root, served through checked URLs.
- Rate limit login (5 attempts per 15 minutes), registration, password reset, and contact endpoints.
- Payment webhooks must verify the gateway signature and the paid amount before changing any status.
- Admin actions on payments, enrollments, and roles are written to the audit log.
- Dependencies are audited monthly (npm audit); critical vulnerabilities are fixed within 7 days.
- Personal data is exported only by Super Admin or Admin roles, and exports are logged.

5. Database Rules

- All schema changes go through migrations; never edit the production database by hand.
- Every table has id, created_at, and updated_at. Use soft delete (deleted_at) for users, courses, and blog posts.
- Foreign keys and indexes are required on relationship fields (user_id, course_id, batch_id) and on columns used for search.
- Money is stored as integers (smallest unit or whole PKR), never as floats.
- Seat updates use database transactions so two students cannot take the last seat at the same time.

6. API Rules

- Versioned routes (/api/v1); nouns for resources, HTTP verbs for actions.
- Consistent response format and correct status codes (200, 201, 400, 401, 403, 404, 409, 422, 500).
- Pagination on all list endpoints (default 20 items, maximum 100).
- Error messages are helpful to users but never expose stack traces or internal details.
- All endpoints are documented (OpenAPI/Swagger) and kept up to date.

7. UI, UX, and Content Rules

- Follow the Design document for colors, fonts, spacing, and components; no one-off styles.
- Mobile-first: every page is built and tested at 360px width first.
- Every form shows inline validation, clear error text, a loading state, and a success message.
- Every image has alt text; every button and link has a clear label.
- Text is written in plain, friendly English; fees, dates, and durations are shown the same way on every page.
- No page may require more than three clicks to reach the Enroll action from a course page.

8. SEO Rules

- One H1 per page that includes the main keyword naturally; headings follow a logical H2/H3 order.
- Every public page has a unique meta title (under 60 characters) and meta description (under 160 characters), editable in admin.
- URLs are short and readable, e.g. /courses/seo-course; changed URLs get 301 redirects.
- Sitemap.xml and robots.txt are generated automatically; canonical tags on every page.
- Schema markup: Organization, Course, FAQPage, and Review where applicable.
- Images are compressed (WebP where possible) and lazy-loaded; Core Web Vitals are monitored.

9. Testing Rules

Type	What is tested	When
Unit tests	Business logic: seat checks, enrollment status changes, fee calculations	On every pull request
API tests	Auth, enrollment, payment approval, role restrictions	On every pull request
End-to-end	Register, enroll, pay by receipt, admin approves, student sees confirmed	Before every release
Cross-browser and device	Chrome, Safari, Edge, Firefox; Android and iPhone	Before launch and major releases
Security check	Access control, upload limits, rate limits, injection attempts	Before launch
Performance	PageSpeed and load test on key pages	Before launch

10. Definition of Done

A task is done only when all of the following are true:

- The feature meets its acceptance criteria in the PRD.
- Code is reviewed, merged, and passes lint and tests in CI.
- It works on mobile and desktop and matches the Design document.
- Errors and empty states are handled.
- Documentation or the admin guide is updated if behavior changed.
- The owner has seen it working on the staging site.

11. Communication Rules

- Weekly progress update: done, in progress, blocked, and next week's plan.
- Any change to scope, cost, or timeline is agreed in writing before work starts.
- Questions to the owner are batched and clearly written; the owner replies within 2 working days.

- Content (text, images, fees, instructor bios) is delivered through one shared folder.